Hoofdstuk 3

Waarom een datawarehouse?

Datawarehouses en Business Intelligence gaan vaak hand in hand. Maar waarom heb je een datawarehouse nodig als je aan Business Intelligence wilt doen? Een datawarehouse is in feite niets anders dan een verzamelbak van gegevens, meestal van gegevens die je in andere databases al beschikbaar hebt. In dit hoofdstuk leer je waarom een datawarehouse nuttig is. In de volgende hoofdstukken leer je hoe een datawarehouse op te zetten.

3.1 Het datawarehouse

In hoofdstuk 1 is als definitie voor het datawarehouse gegeven:

> Een **datawarehouse** is een centrale database gevuld met gegevens uit één of meer aparte bronnen met als doel het maken van rapportages en het doen van data-analyse.

Een datawarehouse is gewoon een relationele database. Dat wil zeggen dat het een database is waarbij de gegevens zijn opgeslagen in de vorm van tabellen. Door middel van de taal SQL (Structured Query Language) kunnen die gegevens gemanipuleerd worden. Het verschil met een 'gewone' database is dat het datawarehouse niet specifiek voor een applicatie is gemaakt, maar dat het een verzamelbak is van gegevens uit allerlei verschillende brondatabases. Omdat alle relevante gegevens worden verzameld in het datawarehouse, kan het dienen als bron voor alle rapportages en analyses. Daarmee wordt het de *'single version'*. Elk rapport haalt zijn gegevens uit dezelfde database en 'vertelt' daarmee hetzelfde verhaal.

> Het datawarehouse fungeert als *'single version of the truth'*.

Dat een datawarehouse de enige bron is voor alle rapportages en analyses is relatief makkelijk bereikt. Zorg dat alles wat van belang is erin zit en dat iedereen altijd alleen het datawarehouse gebruikt voor het verkrijgen van informatie. Dit is overigens een stuk makkelijker gezegd dan gedaan.

In de definitie zit ook nog het woord *'truth'*, waarheid. Het datawarehouse bevat altijd de waarheid. Je moet je echter afvragen of je in staat zult zijn 'de waarheid' te vangen in het datawarehouse. De gegevens die je beschikbaar hebt om te gebruiken zijn vaak onvolledig en van slechte kwaliteit. Met die gegevens wil je in het datawarehouse 'de waarheid' opslaan.

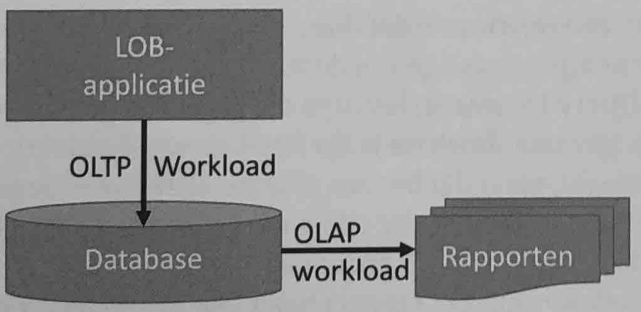
De volledige waarheid is daarmee misschien buiten je bereik. Dat hoeft niet erg te zijn. Het belangrijkste is dat de organisatie de informatie die uit het datawarehouse komt accepteert als de versie van de waarheid waarop wij vertrouwen en sturen.

3.1.1 Waarom een datawarehouse

Het voorgaande is leuk, maar beantwoordt de vraag waarom je een datawarehouse nodig hebt nog niet. Deze vraag laat zich het makkelijkst beantwoorden door te kijken naar de problemen in de informatievoorziening binnen een organisatie als er geen datawarehouse is. Zonder datawarehouse zou de architectuur van je BI-oplossing eruitzien zoals in figuur 3.1. Onderstaand volgt een lijst van mogelijke problemen die optreden als je rapporten en analyses maakt direct op de bronsystemen zoals ERP- en CRM-systemen. In de volgende paragrafen worden deze punten verder uitgewerkt.

- Performance van query's laat te wensen over.
- Het schrijven van de juiste query's is moeilijk en duurt lang.
- Door de rapportage/analyseworkload gaat de performance van de operationele applicaties achteruit.
- De kwaliteit van de gegevens in databases laat te wensen over.
- Verschillende databases bevatten informatie die niet onderling overeenkomt.
- Geschiedenis gaat vaak verloren in operationele databases.

De hier opgesomde punten zijn samen te vatten in performancegerelateerde problemen en inhoudelijke problemen. We beginnen met de performanceproblemen, de inhoudelijke volgen behandelen we daarna.



Figuur 3.1 *Business Intelligence-architectuur zonder datawarehouse*

3.2 Performance van rapporten

Databases die gebruikt worden voor operationele applicaties, zoals CRM-databases, zijn over het algemeen geoptimaliseerd voor de workload (belasting) die de applicatie genereert. Dit type workload noemen we een OLTP-workload. OLTP staat voor OnLine Transaction Processing. Online heeft hier niets te maken met het internet en 'online' zijn. De term komt nog uit de tijd dat het speciaal was dat je rechtstreeks tegen een database aan kon praten. Online staat dan ook voor het feit dat je een realtimeconnectie hebt met de database en interactief met de database kunt werken.

OLTP staat voor Online Transaction Processing, waarbij het gebruik van de database zich kenmerkt door veel kleine acties waarbij een relatief groot aantal van die acties wijzigingen betreft.

Voorbeeld
Denk bijvoorbeeld aan een telemarketeer. Op het moment dat hij een (potentiële) klant aan de lijn heeft, moeten snel de gegevens van die klant op het scherm getoond worden. Het betreft dan het lezen van de gegevens van slechts één klant. Als het om een nieuwe klant gaat, moeten de gegevens ingevoerd worden. Dat houdt in dat er in de database geschreven moet worden. In alle gevallen wordt er een contactnotitie geschreven zodat de organisatie weet wanneer voor het laatst contact is geweest met deze klant en welke afspraken toen zijn gemaakt. Met een beetje geluk moet er ook een nieuwe offerte of zelfs order aangemaakt worden.

Alle hier genoemde acties betreffen zeer kleine hoeveelheden gegevens en er wordt veel geschreven in de database. De database is geoptimaliseerd om deze acties snel en efficiënt uit te voeren.

3.2.1 Normaliseren

Zoals gezegd wordt een database meestal geoptimaliseerd voor het soort gebruik waarvoor hij is gemaakt. Bij veel applicaties is de belasting een OLTP-workload. Databases die een OLTP-workload ondersteunen, worden in meer of mindere mate genormaliseerd.

Als je een database gaat maken, moet je beslissen welke tabellen je nodig hebt. Een tabel om klantgegevens in op te slaan, een tabel met producten, de orderstabel enzovoort. Voor OLTP-databases wordt over het algemeen gekozen voor genormaliseerde databases. Het proces van databaseontwerp heet dan normaliseren en het resultaat is (meestal) dat de database in de zogenaamde derde normaalvorm staat.

- Normaliseren van een database voorkomt redundantie en vergroot de consistentie van de gegevens.

Belangrijk bij normaliseren is het voorkomen van redundantie. Redundantie is het dubbel opslaan van gegevens. Stel dat jouw adres tweemaal is opgeslagen in dezelfde database. Je verhuist en geeft je adreswijziging door. Die adreswijziging moet nu op twee plekken doorgevoerd worden. Dat is grofweg tweemaal zo duur (met duur zullen we in dit boek meestal refereren aan tijd) dan wanneer diezelfde wijziging maar eenmaal weggeschreven moet worden. Het schrijven in een database is dus sneller als er minder redundantie in zit. Bijkomende voordelen zijn dat de kans op fouten (op de ene plek is de adreswijziging wel doorgevoerd, op de andere niet) kleiner wordt en dat de database als geheel wat kleiner blijft.

- Het schrijven in een database gaat sneller als er minder redundantie in zit.

Als je voornamelijk leest, is het dubbel voorkomen van gegevens helemaal niet zo erg. Op hoe meer plekken de gegevens staan, hoe groter de kans dat je ergens snel bij kunt. Rapportage- en analysesystemen lezen eigenlijk alleen maar in de database. Een alleen-lezen workload wordt ook wel een OLAP-workload genoemd. OLAP staat voor OnLine Analytical Processing. Behalve de verkleinde kans op fouten (inconsistenties) heeft een OLAP-workload geen voordeel van normaliseren.

OLAP staat voor Online Analytical Processing, waarbij het gebruik van de database zich kenmerkt door voornamelijk lezen.

Normaliseren heeft een aantal effecten:
- Er ontstaan veel verschillende tabellen.
- De tabellen worden smal (hebben relatief weinig kolommen).

Er ontstaan veel verschillende tabellen

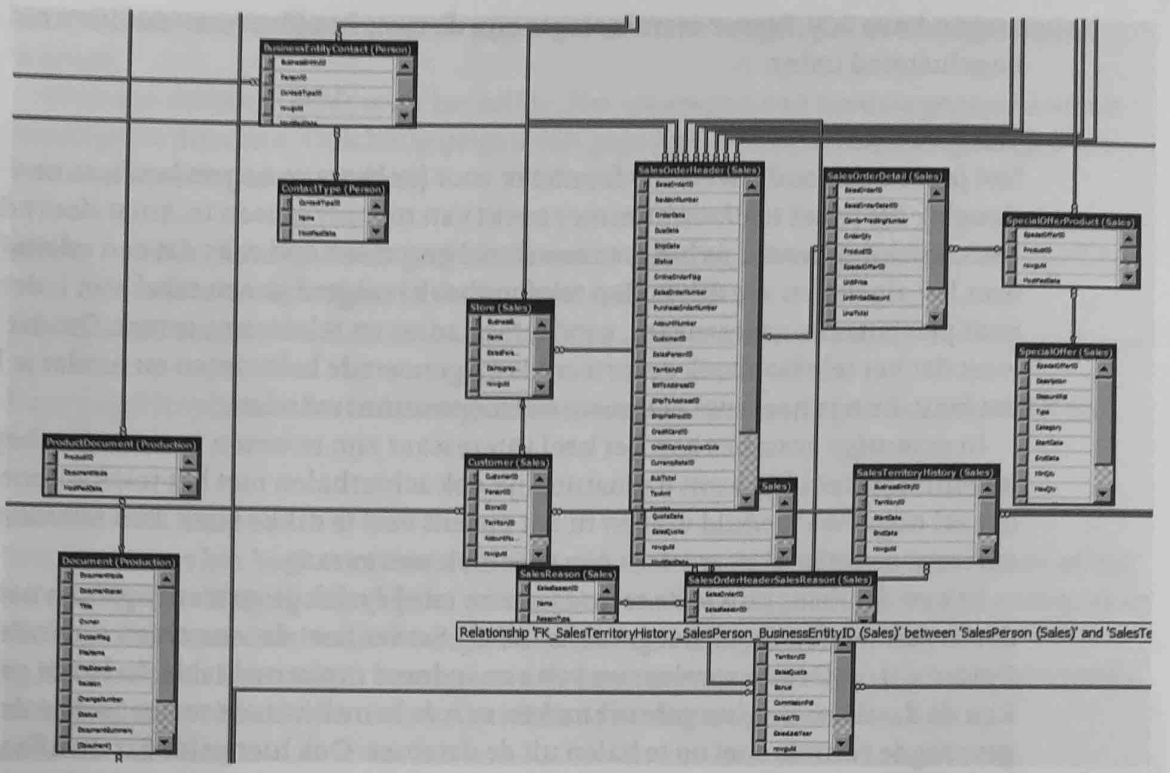
Een effect van normaliseren is dat er veel verschillende tabellen ontstaan. Om redundantie te voorkomen worden (potentieel) dubbele gegevens in aparte tabellen gezet. Je kunt dan bijvoorbeeld denken aan postcodes en woonplaatsen. Als je een postcode weet, volgt daar uniek de bijbehorende woonplaats uit. In plaats van bij elke persoon in je database zowel postcode als woonplaats op te slaan, kun je ervoor kiezen alleen de postcode op te nemen bij die persoon. In een aparte tabel staan dan de woonplaatsen met de postcodes en op basis van postcode zoek je in deze tabel de juiste woonplaats op.

Zoals gezegd ontstaan er dus veel verschillende tabellen. In een SAP-database gaat het al snel om 40.000 of meer verschillende tabellen. In figuur 3.2 zie je een voorbeeld van een ERD (Entity Relationship Diagram) van een eenvoudige voorbeelddatabase.

De tabellen worden smal

Een tweede effect van normaliseren is dat de tabellen vrij 'smal' worden. Met smal wordt hier bedoeld dat een tabel uit relatief weinig kolommen bestaat. Er zit niet zo heel veel informatie in een tabel. Het voordeel voor een OLTP-workload is dat één record nu klein is (weinig bytes) wat het efficiënt maakt een record te wijzigen of een nieuw record toe te voegen. Een nadeel voor OLAP-workloads is dat er bijna geen rapporten bestaan die wat betreft de benodigde gegevens genoeg hebben aan één tabel. Er zullen joins geschreven moeten worden (gegevens uit meer tabellen moeten worden samengevoegd) en dat kan soms heel lastig zijn. Bovendien resulteren query's die heel veel tabellen aan elkaar joinen (die veel tabellen gebruiken) vaak in slechte performance.

Een nadeel van normaliseren voor OLAP-workloads is dat de benodigde query's moeilijk en traag worden.



Figuur 3.2 *Genormaliseerde database*

Samenvattend kun je concluderen dat genormaliseerde databases goed zijn voor OLTP-workloads en daarmee voor LOB(Line Of Business)-applicaties zoals CRM-systemen en boekhoudpakketten. Voor het maken van rapportages en het doen van analyses die grote hoeveelheden gegevens nodig hebben, zijn genormaliseerde databases heel slecht voor de performance. Aangezien voor het succes van een Business Intelligence-oplossing de acceptatie van de gebruikers van kritiek belang is, en gebruikers over het algemeen een hekel hebben aan slechte performance, is het rechtstreeks gebruikmaken van de operationele databases dus een slecht idee. Met het bouwen van een datawarehouse heb je de kans dezelfde gegevens op een andere manier, in een andere tabelstructuur, op te slaan. Uiteraard kies je dan voor een tabelstructuur die de performance van de Business Intelligence-workload goed ondersteunt (zie hoofdstuk 4).

Met het bouwen van een datawarehouse heb je de mogelijkheid de gegevens op te slaan in een structuur die geschikt is om snel en adequaat informatie uit een database te lezen.

3.2.2 Indexen

In de vorige paragraaf is besproken dat de tabelstructuur van een database (mede) bepalend is voor de performance van die database. Naast de tabelstructuur zijn er nog andere factoren van invloed op de performance. Een belangrijke daarvan is de gekozen indexstrategie. Afhankelijk van het databaseplatform zijn er veel verschillende soorten indexen. Het gaat hier te ver alle soorten indexen te bespreken, maar gebruik-

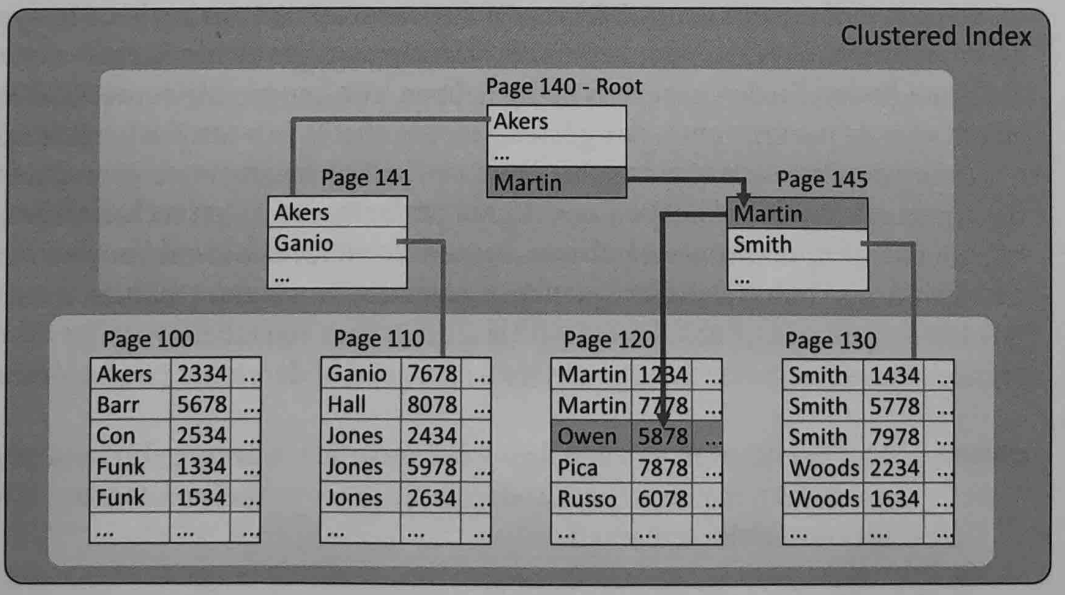
makend van SQL Server-terminologie zijn de twee hoofdvormen de clustered en de nonclustered index.

Clustered index

Stel je eens een ouderwets telefoonboek voor (ze bestaan nog en het heet niet Google). Als je het telefoonnummer zoekt van meneer Jansen in Amersfoort die op de Leusdenseweg woont, is het antwoord snel gevonden ondanks dat een telefoonboek over het algemeen erg dik is. Een telefoonboek is eigenlijk een tabel met kolommen voor plaatsnaam, achternaam, voorletters, adres en telefoonnummer. Omdat je echter weet dat het telefoonboek gesorteerd is op genoemde kolommen en omdat je het alfabet kent, kun je heel snel het juiste telefoonnummer vinden.

In sommige situaties kan het heel interessant zijn te weten van wie een bepaald telefoonnummer is. Je kunt dat natuurlijk ook achterhalen met het telefoonboek – maar dat zal nogal wat geduld vragen in het opeens veel te dikke boek. Een telefoonboek is extreem geoptimaliseerd voor één specifiek soort vraag.

In een database kun je de records in een tabel fysiek gesorteerd opslaan net zoals dat in een telefoonboek het geval is. In SQL Server heet dat een clustered index (zie figuur 3.3), in Oracle spreken we van een indexed organised table. In beide gevallen kan de database-engine gebruikmaken van de kennis van de sorteervolgorde om de gevraagde records snel op te halen uit de database. Ook hier geldt dat dit alleen voor specifieke query's geldt, namelijk query's die 'toevallig' baat hebben bij de gekozen sorteervolgorde.



Figuur 3.3 *Clustered Index*

Een clustered index kan lezen in de database erg snel maken. Er zijn ook nadelen. Stel je eens voor dat je verantwoordelijk bent voor het up-to-date houden van het telefoonboek. Nieuwe inwoners achter in het telefoonboek toevoegen is geen optie. Vanwege de sorteervolgorde van de records moet je de nieuwe regel op de juiste plek invoegen. Het probleem is echter dat de pagina waar je op moet schrijven al helemaal vol is. Je

moet nu een truc bedenken om de gegevens zo toe te voegen dat ze altijd gevonden worden.

Voor een database geldt weer hetzelfde. Het toevoegen van nieuwe gegevens wordt moeilijk en dus duur. Ook het wijzigen van gegevens kan opeens grote gevolgen hebben en daardoor duur worden.

> Over het algemeen geldt: indexen maken het lezen uit een database sneller maar het schrijven in de database langzamer.

Deze regel impliceert dat indexen goed zijn voor OLAP, maar minder aantrekkelijk voor OLTP.

Nonclustered index

Stel je nu eens het boek voor dat je op dit moment leest. Je hebt misschien gehoord dat statistieken belangrijk zijn bij het gebruik van indexen en je vraagt je af of er ergens in dit boek iets staat over statistieken. Je kunt nu twee dingen doen.

1. Het boek van kaft tot kaft doorlezen in de hoop ergens iets over statistieken te vinden.
2. Achter in het boek kijken en statistieken opzoeken in de index.

De eerste strategie is zinvol als je studeert voor je tentamen. Voor het beantwoorden van de gestelde vraag is de tweede strategie een stuk sneller. Je bent er wel van afhankelijk of de schrijver de moeite heeft genomen een index te maken.

De index in een (studie)boek bestaat uit op alfabet gesorteerde steekwoorden gevolgd door een verwijzing in de vorm van een paginanummer. Als je het woord statistieken niet ziet staan, weet je dat je een ander boek moet zoeken om de vraag beantwoord te zien. Als je het wel tegenkomt, hoef je het boek alleen nog maar open te slaan op de juiste pagina.

De index in een boek maakt het boek dikker. Het boek is daardoor ook iets duurder. De indexen hebben alleen meerwaarde als je het boek regelmatig als naslagwerk gebruikt om nog eens een onderwerp terug te lezen. Met andere woorden, alleen een bepaald soort gebruik heeft baat bij indexen.

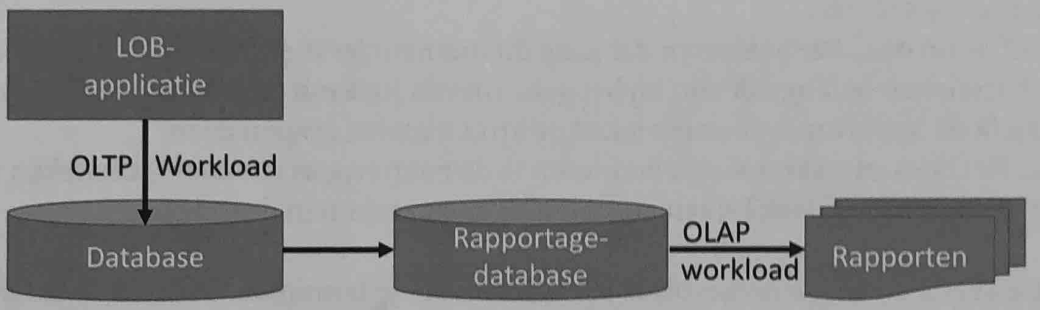
Een index zoals in een studieboek is te vergelijken met een nonclustered index in een database (waar nonclustered de SQL Server-term is). Ook hier geldt, net zoals bij de clustered index, dat sommige vragen baat hebben bij de index, maar andere niet. Op het moment dat er wijzigingen zijn, dat er geschreven moet worden in de database, maken indexen dat duurder, ofwel trager.

Conclusie

Indexen zijn een belangrijke performance tuning feature voor databases. Hoewel de praktijk heel genuanceerd is en er altijd uitzonderingen zijn, geldt in hoofdlijnen: indexen maken een database sneller voor lezen (goed voor OLAP-workloads) maar trager voor schrijven (slecht voor OLTP-workloads). Je kunt dus niet zomaar indexen gaan maken in operationele databases om je Business Intelligence-oplossing sneller te maken.

Als je performanceproblemen hebt met query's in rapporten, heb je misschien wel een aparte rapportagedatabase nodig (zie figuur 3.4). Je kunt een rapportagedatabase maken door elke nacht een kopie van de productiedatabase te maken en daar de rapporten op te baseren. Er zijn uiteraard ook geavanceerdere methoden die twee databases met elkaar kunnen synchroniseren. De ultieme rapportagedatabase is een datawarehouse. Met een rapportagedatabase verandert de architectuur van figuur 3.1 in die van figuur 3.4.

Een datamart of een rapportagedatabase kan indexen bevatten die speciaal voor rapporten en analyses zijn gemaakt. Dit levert verbeterde performance op ten opzichte van de bronsystemen en daarmee betere acceptatie.



Figuur 3.4 *Business Intelligence-architectuur met rapportagedatabase*

3.3 Schrijven van query's

In de vorige paragraaf heb je gelezen waarom een operationele database niet geschikt is voor Business Intelligence-doeleinden. Zowel qua tabelstructuur als qua indexstrategie is de database geoptimaliseerd voor de OLTP-workload die de applicatie genereert. De rapportage- en analysequery's die een Business Intelligence-oplossing genereren, hebben daar over het algemeen onder te leiden.

Normaliseren van databases leidt ook tot het tweede probleem zoals beschreven in de opsomming aan het begin van dit hoofdstuk: het schrijven van de juiste query's is moeilijk en duurt lang.

In paragraaf 3.2 staat al dat een genormaliseerde database veel joins in query's tot gevolg heeft en dat de performance van de query's daaronder te lijden heeft. Dit heeft echter ook een tweede indirect performanceverlies als gevolg.

In veel organisaties worden vanuit de organisatie verzoeken om informatie verstuurd aan ICT. Veel bedrijven hebben zelfs een aparte afdeling die dit soort verzoeken afhandelt. Soms is dat door een query te schrijven en de resultaten in de vorm van een Excel-werkboek aan de aanvrager te sturen. In andere gevallen wordt er misschien wel een rapport gemaakt met bijvoorbeeld SQL Server Reporting Services, dat vervolgens aan de hele organisatie beschikbaar wordt gesteld.

Het verborgen performanceprobleem zit hier in de tijd die verstrijkt tussen het aanvragen van de informatie en het terugkrijgen daarvan. Misschien dat de uiteindelijke query nog geen seconde doorlooptijd had. Maar toen de aanvraag werd gedaan had

de afdeling het druk met iets anders, de dag erop was er iemand ziek, daarna was het weekend en al met al heeft het een week geduurd voordat de gevraagde informatie er is.

■ Vaak gaat er tijd verloren tussen het ontstaan van een informatiebehoefte en het invullen daarvan.

In paragraaf 1.1.1 hebben we als soort werkdefinitie voor Business Intelligence gegeven: *Making Better Decisions Faster.* Een andere oneliner, uit de Microsoft-marketingmachine, is: *answers at the speed of thought.* Beide oneliners doelen op het ideaalbeeld dat je antwoord krijgt op je vraag op het moment dat je de vraag bedenkt. In een op-de-hei-brainstormsessie van het management moet iedereen van alles kunnen roepen en meteen moeten de relevante feiten getoond kunnen worden. Dit soort ideeën (beloftes?) worden duidelijk niet gerealiseerd als we verzoeken moeten indienen bij een aparte afdeling.

3.3.1 Wie maakt de rapporten?

Bovenstaand probleem zou met self-service BI misschien wel opgelost worden. Waarom iemand anders vragen als je het zelf ook kan? Op zich een heel terechte vraag, maar kun je het zelf ook? Denk nog eens aan SAP met meer dan 40.000 tabellen. Die tabellen hebben over het algemeen geen mooie namen als klant en order. Is iemand die op de financiële administratie werkt en voornamelijk bezig is met lijsten van grootboekposten in Excel, in staat om op zo n complexe database een zinvolle query te schrijven?

Als we daadwerkelijk sneller bij de gewenste informatie willen komen, moeten we de mensen die informatie nodig hebben ook in staat stellen bij die informatie te komen. Daar zijn minimaal twee zaken voor nodig:

1. De mensen moeten de juiste tools krijgen.
2. De beschikbare gegevens moeten op een begrijpelijke manier worden aangeboden.

Met begrijpelijk bedoelen we dan niet begrijpelijk voor een SAP-consultant met minimaal vijf jaar ervaring! Een datawarehouse, of in vele gevallen een datamart, kan de gegevens beter inzichtelijk aanbieden aan de gebruikers dan de operationele systemen doen.

■ Datamarts bieden informatie aan gebruikers aan in een voor de gebruiker begrijpelijke eenvoudige vorm.

3.4 Rapportage-impact op het primaire proces

Het derde genoemde probleem uit paragraaf 3.1 met rapporteren/analyseren op operationele databases, is dat de performance van de operationele applicaties achteruitgaat door die rapportages en analyses.

Tot nu toe hebben we gekeken naar de te verwachten performanceproblemen als je direct op de operationele database gaat rapporteren. Het databaseontwerp sluit niet goed aan bij de workload. De indexstrategie is niet optimaal voor rapportages. Maar er is nog meer. Kan de databaseserver de extra workload van de Business Intelligence-oplossing wel aan? Daarnaast bijten de OLTP- en OLAP-workload elkaar misschien wel als ze gelijktijdig met dezelfde gegevens willen werken. Er zijn dus nog twee mogelijke problemen:

1. Gebruik van hardware resources.
2. Gelijktijdig gebruik van dezelfde gegevens.

3.4.1 Resource-gebruik

Uiteraard is het zo dat een server het drukker krijgt als je er meer van vraagt. In eerste instantie gaat de CPU-belasting omhoog. Kan de server deze extra belasting nog wel aan? Zo ja, heeft dat impact op de performance van de applicatie die gebruikmaakt van de database? En kun je je dat eventuele performanceverlies wel permitteren, nog afgezien van de performance van je eigen rapportages en analyses?

Bij veel databaseservers is CPU niet de eerste beperkende factor. Geheugen en storage (harde schijven, SAN's) zijn eerder bottlenecks.

Een **SAN (Storage Area Network)** is een populair opslagmedium dat veel gebruikt wordt voor databases.

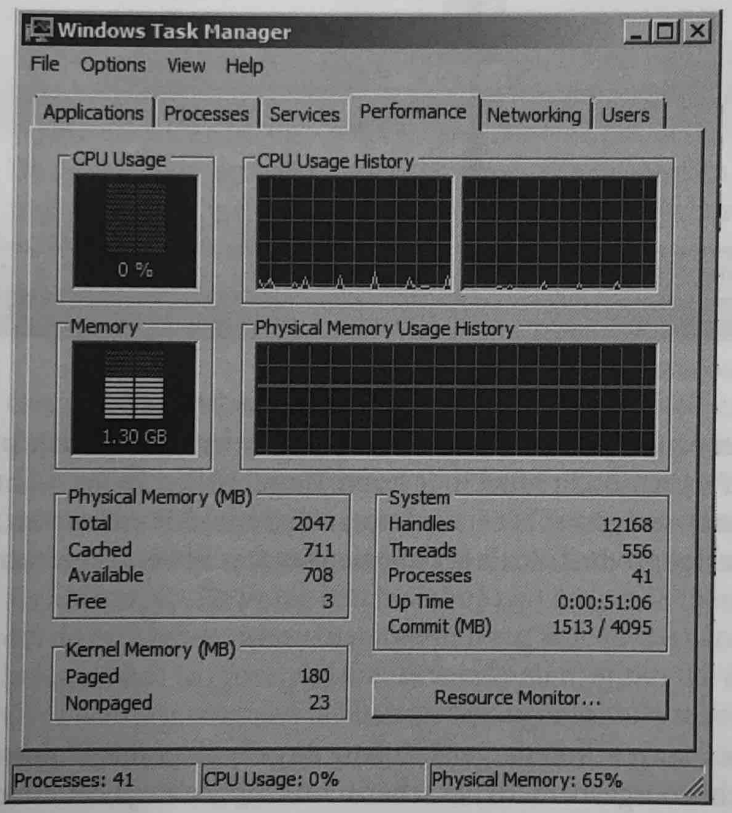
De meeste databasesystemen proberen zo veel mogelijk gegevens in het geheugen te houden. Als iets eenmaal van de harde schijf is ingelezen in het geheugen, blijft het daar, tenzij het echt niet anders kan. De gedachte daarachter is dat als dezelfde gegevens later weer gebruikt worden, ze rechtstreeks uit het geheugen gehaald kunnen worden in plaats van dat de data weer van de harde schijf moet worden ingelezen. Inlezen vanuit geheugen is vele malen sneller dan lezen vanaf schijf. Met SSD (Solid State Disk) wordt dat verhaal anders.

Analyses waar grote gegevensverzamelingen voor nodig zijn verstoren dit proces. Je leest misschien tien jaargangen aan oude orders in om trends op omzet te analyseren. Deze data wordt nu ingelezen en komt in het geheugen van de databaseserver terecht. Als de server heel veel geheugen heeft, is dat wellicht geen probleem. Maar met minder geheugen zou het kunnen zijn dan de databaseserver andere gegevens eerst uit het geheugen moet weggooien (flushen). Dat is op zich niet erg. Maar er is wel een kans dat die gegevens enkele seconden later al weer nodig zijn en dan vanaf schijf ingelezen moeten worden. Ze zitten immers niet meer in het geheugen. En van schijf inlezen is traag.

Grote gegevensverzamelingen kunnen voor het OLTP-proces belangrijke gegevens uit het geheugen verdringen waardoor het OLTP trager wordt.

Als dergelijke processen optreden zul je zien dat de databaseserver meer IO gaat doen. IO staat voor Input/Output, dus het echt lezen van en schrijven naar de harde schijven.

Er is een gerede kans dat een Business Intelligence-workload een bottleneck veroorzaakt op de databaseserver als je niet een aparte server (met een datawarehouse) inricht voor de BI-processen. De geïntroduceerde bottleneck kun je je vaak niet permitteren omdat het primaire bedrijfsproces hiermee verstoord wordt. In figuur 3.5 zie je een voorbeeld waar al 1.3 GB geheugen van de slechts 2 GB beschikbare geheugen in gebruik is. Er is dus niet veel meer nodig om het gehele geheugen te vullen. Dit zijn overigens geen representatieve getallen voor databaseservers.

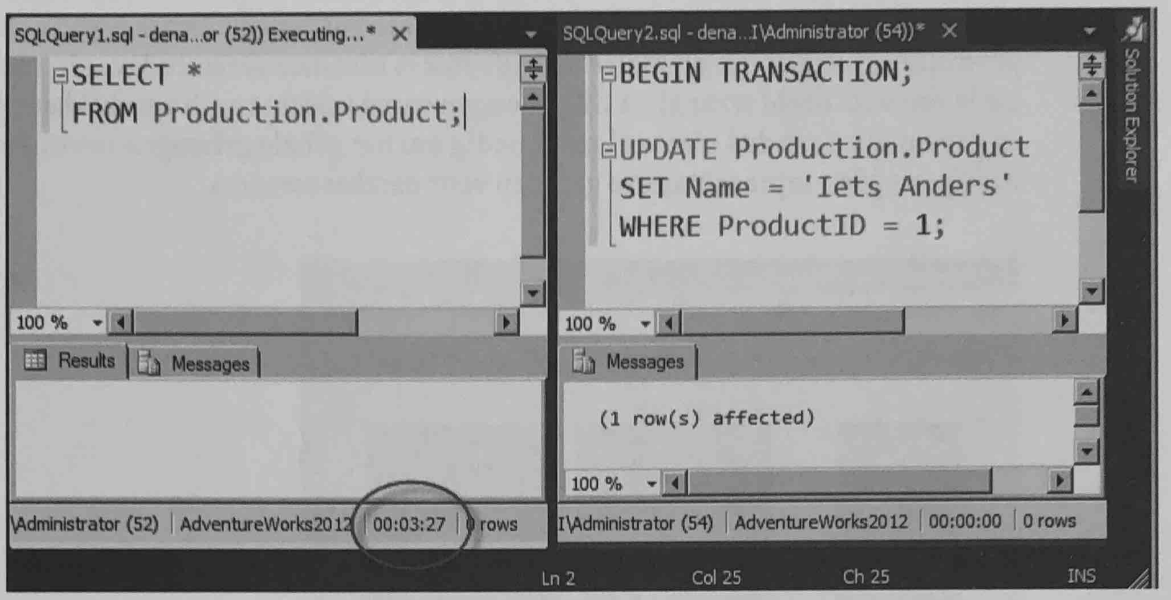


Figuur 3.5 *Geheugenverbruik*

3.4.2 Concurrency

Concurrency is het tegelijkertijd werken in de database, ofwel met meer mensen en/of processen tegelijkertijd werken. Een DBMS (DataBase Management System) moet hier rekening mee houden en de gebruikers beschermen voor vervelende effecten. Wat bijvoorbeeld als één gebruiker een record wijzigt en een andere gebruiker precies op dat moment datzelfde record wil lezen? Moet dan de nieuwe waarde getoond worden, zelfs als gebruiker één zich nog kan bedenken? Of zullen we voor de zekerheid de oude waarde tonen voor zolang de eerste gebruiker zijn wijziging nog niet heeft bevestigd (commit)? Een derde optie is de tweede gebruiker, de lezer, te laten wachten tot de eerste gebruiker helemaal klaar is met zijn wijziging.

Concurrency is het met meer gebruikers tegelijkertijd werken in dezelfde database.



Figuur 3.6 *Locking door concurrency*

In figuur 3.6 zie je hier een voorbeeld van. In het scherm links in het plaatje probeert iemand de hele tabel Production.Product in te lezen. De gebruiker in het rechterscherm is echter net iets eerder met het starten van een transactie en het van naam veranderen van het eerste product. Zoals je kunt zien staat er in het rechterscript bovenaan:

```
BEGIN TRANSACTION
```

De bijbehorende `COMMIT` staat echter nergens. Zolang de gebruiker niet commit, is de wijziging nog niet echt doorgevoerd. In het scherm linksonder zie je het effect op de lezer. Het linkerscherm zit al 3 minuten en 27 seconden te wachten op de commit in het rechterscherm, zodat duidelijk is wat nu de correcte naam van product 1 is. Het linkerscherm kan niet verder totdat het rechterscherm zijn transactie afmaakt.

Het bovenstaande is een voorbeeld van hoe SQL Server default omgaat met concurrency. Dit is geen ongewoon scenario als je rapportages op een operationele database loslaat. De OLTP-workload van bijvoorbeeld een CRM-applicatie kenmerkt zich door veel transacties die vergelijkbaar zijn met het rechtervenster. In rapporten worden vaak wat grotere datasets (gegevensverzamelingen) opgehaald, vergelijkbaar met het linkervenster. (Overigens is het gedrag van SQL Server te beïnvloeden en gedragen andere systemen, zoals Oracle, zich weer anders.)

De conclusie is dat het nog al wat consequenties kan hebben als grote rapportages en analyses query's uitvoeren op operationele databases. Let op het woordje 'kan'. Als je in een relatief kleine omgeving werkt met voldoende hardware resources (CPU en geheugen) hoeven zich geen problemen voor te doen. Misschien kun je query's wel 's nachts laten uitvoeren als het operationele proces zo goed als tot stilstand is gekomen.

Probeer voordat je rechtstreeks op een operationele database grote query's gaat uitvoeren een inschatting te maken van de effecten en kijk of deze acceptabel zijn.

Rapportages en analyses kunnen grote impact hebben op de performance van operationele processen. De rapportages en analyses loslaten op een datawarehouse lost dit probleem op.

3.5 Kwaliteitsproblemen

Aan het begin van deze paragraaf heb je kunnen lezen dat er een aantal redenen is waarom het nuttig kan zijn een datawarehouse te bouwen. Grofweg vielen die in twee categorieën uiteen: performancegerelateerde problemen en kwaliteitsproblemen. De performancegerelateerde problemen zijn besproken in de paragrafen 3.2 tot en met 3.4. De (mogelijke) problemen die nog overblijven zijn:

- De kwaliteit van de gegevens in databases laat te wensen over.
- Verschillende databases bevatten informatie die niet onderling overeenkomt.
- Geschiedenis gaat vaak verloren in operationele databases.

Om met het eerste probleem te beginnen: de kwaliteit van de gegevens in productiedatabases laat te wensen over. Er bestaan bedrijven die niets anders doen dan proberen dit probleem voor hun klanten op te lossen. Dat geeft wel aan dat het een veelvoorkomend probleem is. Dit soort bedrijven draait een standaard set van query's om de meest voorkomende zaken op te sporen en lost de problemen dan zo goed en kwaad als het kan op. De vraag is echter: hoe zijn die problemen ontstaan en hoe voorkom je dat de problemen niet vanzelf weer terugkomen? Het antwoord op die laatste vraag is helaas: dat is (zo goed als) niet te voorkomen. Bijna elke database kent vervuiling. Over welke problemen hebben we het eigenlijk? De meestvoorkomende zijn:

- dubbele gegevens;
- ontbrekende gegevens;
- foutieve gegevens;
- inconsistente gegevens.

We zullen ze een voor een kort bespreken.

3.5.1 Dubbele gegevens

Een veelvoorkomend probleem is dat databases vaak dubbele gegevens bevatten. Kijk bijvoorbeeld eens naar onderstaande tabel:

LeverancierNr	Bedrijfsnaam	ContactPersoon	Email	RekeningNr
1	TrainSQL	Peter ter Braake	peter@trainsql.nl	12345
2	Train-SQL	Janneke Brummelhuis	Janneke@trainsql.nl	54321
3	TrainSQL B.V.	Peter ter Braake	peter@trainsql.nl	12345
...	...	...	...	...

Figuur 3.7 *Tabel met dubbele gegevens*

Iemand met een beetje inzicht vermoedt hier al snel dat het om dezelfde leverancier gaat. Bij record 1 en 3 is alleen de bedrijfsnaam anders. Bovendien is het enige verschil de toevoeging bv. Record 2 is nog iets lastiger omdat andere gegevens ook anders zijn. Tegelijkertijd, met dezelfde domeinnaam in het e-mailadres, is het toch weer heel waarschijnlijk dat het hier om dezelfde leverancier gaat.

Voor het operationele systeem zijn dergelijke dubbele records waarschijnlijk geen groot probleem. Verschillende bestellingen die eigenlijk bij dezelfde leverancier vandaan komen staan geregistreerd bij verschillende leveranciers, maar zolang de bestelde goederen of diensten geleverd worden en de leverancier betaald krijgt, is er niet veel aan de hand.

Vanuit een Business Intelligence-perspectief is er echter wel degelijk een probleem. Als je query's schrijft om te achterhalen hoeveel leveranciers er zijn en hoeveel bestellingen per leverancier besteld zijn, komen er totaal verkeerde resultaten uit. Misschien concludeer je wel dat er een kostenbesparing mogelijk is door alle bestellingen meer te concentreren bij enkele preferred suppliers. Door deze meer business te gunnen, kun je hogere kortingen afdwingen. Achteraf blijkt dat deze actie bij voorbaat gedoemd was te mislukken omdat er helemaal niet zoveel leveranciers bleken te zijn.

■ Dubbele records in operationele systemen leiden tot foutieve resultaten tijdens analyses.

Voorkomen van dubbele records is bijna onbegonnen werk in databases waar met grote regelmaat veranderingen in plaatsvinden. Als er net een nieuw iemand is aangenomen op de administratie, iemand die nog niet de inhoudelijke kennis heeft van wie de leveranciers zijn, kan er gemakkelijk een nieuwe aangemaakt worden als de bestaande, bijvoorbeeld door een tikfout, niet gevonden wordt.

Ontdubbelen van gegevens in operationele databases is bijna onbegonnen werk. Het is makkelijker om een datawarehouse te maken en ervoor te zorgen dat er geen dubbele records in het datawarehouse voorkomen. Hoewel het in theorie beter is de vervuiling aan te pakken bij de bron, de operationele database, werkt het in de praktijk beter dit in het datawarehouse te doen. Over het datawarehouse heb je namelijk veel meer controle.

■ Doordat een datawarehouse gecontroleerd gevuld wordt door zorgvuldig geteste processen, is het voorkomen van dubbele gegevens in datawarehouses makkelijker dan in operationele databases.

3.5.2 Ontbrekende gegevens

Naast dubbele records is het ontbreken van gegevens vaak een probleem. Stel je doet een analyse om te kijken waar je in Nederland je omzet scoort. Je zou deze analyse kunnen doen op basis van postcode. In figuur 3.8 zie je waar dat toe kan leiden: een groot deel van de omzet valt onder NULL, ofwel onbekend. Er blijken records in het systeem te zitten waarbij de postcode niet is ingevuld.

```
OntbrekendePostco...TER-PC\Peter (55)) ×
CREATE TABLE Omzet
(
OmzetKey INT IDENTITY(1, 1)
, Postcode VARCHAR(7)
, Omzet MONEY
);

INSERT Omzet
VALUES
    ('3500', 8000)
    , ('1000', 7000)
    , (NULL, 800)
    , (NULL, 3000);

SELECT
    Postcode
    , SUM(Omzet) AS Totaal
FROM dbo.Omzet
GROUP BY Postcode;
```

100 %

Results | Messages

	Postcode	Totaal
1	NULL	3800,00
2	1000	7000,00
3	3500	8000,00

Figuur 3.8 *Ontbrekende gegevens*

Als er heel veel records zijn en voor slechts een klein percentage van die records geldt dat de postcode ontbreekt, is het effect op de analyse misschien verwaarloosbaar. Verhoudingsgewijs blijft de rest van de analyse hetzelfde. Maar kun je daar op vertrouwen? Wat nu als alleen postcodes uit de regio Rotterdam ontbreken? Misschien volgt de verkoper uit die regio een andere werkwijze waardoor alleen daar geen postcode wordt ingevuld. De analyse wordt nu echt incorrect.

Een dergelijk voorbeeld met postcodes is helder maar natuurlijk ook erg vereenvoudigd. Een snelle oplossing voor het probleem zou zijn het veld postcode verplicht te maken in de database. De database accepteert geen records meer zolang de postcode niet is ingevuld. Probleem opgelost!

Vaak zitten dit soort eenvoudige oplossingen evenwel de dagelijkse gang van zaken in de weg. Iemand staat bijvoorbeeld in de winkel op het punt een nieuwe televisie te kopen voor 1500 euro. De verkoper vraagt de postcode en de klant weigert deze te geven. Of de klant zegt: ik ben net verhuisd, ik weet mijn nieuwe postcode nog niet uit mijn hoofd.

Slimme verkopers bedenken een truc om de verplichte invoer van de postcode te omzeilen. Ze vullen bijvoorbeeld 9999 XX in. Met een beetje pech bedenkt elke verkoper dezelfde truc, maar iedereen kiest zijn eigen dummywaarde voor postcode.

Het probleem van de ontbrekende postcodes is nu groter geworden in plaats van kleiner. Voorheen was er nog één categorie, NULL, die eruit sprong. Nu staat er omzet onder verkeerde postcodes en staat andere omzet onder niet-bestaande postcodes. En deze omzet is lastiger terug te vinden dan alle omzet waarvan de postcode niet bekend is.

Over het algemeen is het goed om in operationele database door middel van constraints (beperkingen) zo veel mogelijk af te dwingen dat gegevens correct en volledig worden ingevoerd. Een postcodeveld kan verplicht gemaakt worden en de database kan controleren dat de postcode bestaat uit vier cijfers, een spatie en twee letters. Voor de kwaliteit van de gegevens en daarmee van de rapportages, is dat goed. De werkbaarheid van het operationele systeem en daarmee van primaire processen, kon wel eens in gevaar komen. Omdat dat laatste vaak belangrijker wordt gevonden, moet je er altijd van uitgaan dat je gegevens niet compleet zijn.

In operationele databases ontbreken soms gegevens die voor analyses van belang zijn. Tijdens het vullen van een datawarehouse kan dit gedetecteerd worden en kunnen de ontbrekende gegevens worden aangevuld of expliciet gekenmerkt worden als onbekend.

3.5.3 Foutieve gegevens

Hiervoor heb je al een voorbeeld gezien van foutieve gegevens. Volgens Wikipedia bestaat postcode 9999 XX wel, maar is het een fictief adres. Naast postcodes die er correct uitzien maar dat bij nader onderzoek niet blijken te zijn (fictief, niet bestaand), bestaat er een kans dat je postcodes in de database tegenkomt die duidelijk geen postcode zijn. Bijvoorbeeld 123 AB. Deze postcode heeft maar drie cijfers en kan dus geen Nederlandse postcode zijn.

Nog lastiger zijn gegevens die niet zo duidelijk zijn afgebakend. Neem de prijs van een product. In de database staat bijvoorbeeld 1,5 euro. Maar degene die de prijs heeft ingevoerd heeft een tikfout gemaakt, het moest 15 euro zijn. Deze fout zal er waarschijnlijk in het primaire proces wel uitgehaald worden (klagende klanten). Er zijn echter tal van voorbeelden te bedenken van fouten in de gegevens die nauwelijks te achterhalen en te voorkomen zijn.

Initiatieven zoals Data Quality Services van Microsoft helpen om foutieve gegevens tijdens het laden van een datawarehouse op te sporen en (deels) automatisch te verbeteren.

3.5.4 Inconsistente gegevens

Het laatste kwaliteitsprobleem dat we hier kort beschrijven is het probleem van inconsistente gegevens.

In het begin van het computertijdperk bouwde elk bedrijf zijn eigen softwarepakketten. De meeste bedrijven hebben iets van een ordersysteem nodig. Dit systeem werd gemaakt op verzoek van het bedrijf, precies naar de wensen van het bedrijf. Dit zogenaamde maatwerk vinden we tegenwoordig veel te duur. Het is vervangen door wat we noemen off-the-shelf software. Als bedrijf koop je een CRM-applicatie. Hoe minder aanpassingen daar vervolgens aan gedaan moeten worden, hoe beter.

In veel bedrijven zijn de primaire processen vergelijkbaar, maar op de details zijn ze juist anders. Als bedrijf moet je immers ergens je onderscheidend vermogen vandaan halen. Een standaardapplicatie heeft als uitdaging zo flexibel te zijn dat alle afwijkingen op de meest gangbare werkwijze in de applicatie zijn onder te brengen. En het liefst zonder extra maatwerk of met minimaal maatwerk.

Standaardapplicaties geven de vrijheid om voor een organisatie niet-consistente informatie op te slaan in de database.

Stel dat in een bedrijf het een regel is dat de klant die bestelt ook de factuur krijgt. Met andere woorden, het adres van de klant is hetzelfde als het factuuradres. Het afleveradres mag afwijken. De klant mag bepalen waar de goederen afgeleverd moeten worden. Er zijn echter bedrijven waar het adres van de klant die de bestelling doet wel mag afwijken van het factuuradres (stuur de factuur maar naar het hoofdkantoor!). Een standaard CRM-pakket beschikt over de flexibiliteit om bij een order drie verschillende adressen te registreren. Op die manier kan het pakket zo wel alle mogelijk denkbare varianten van het verkoopproces ondersteunen.

Het probleem hiervan is dat het bijna een zekerheid is dat er orders te vinden zijn waarvoor geldt dat het factuuradres anders is dan het adres van de klant die de bestelling plaatste, ook als dat volgens de procesbeschrijving van de organisatie niet kan. Daar waar de database die ruimte biedt, zullen er gegevens ontstaan die daar gebruik van maken. Het kan gaan om uitzonderingen. We doen dat normaliter niet, maar voor deze ene keer... Het kan echter ook om een fout gaan. Je hebt nu inconsistenties in je gegevens.

Een datawarehouse lost deze inconsistenties niet meteen op. Tijdens het laden van het datawarehouse komen ze waarschijnlijk wel aan het licht. Er ontstaat dan een keuze hoe hiermee om te gaan.

3.6 Verschillende operationele systemen

In de vorige paragraaf heb je kunnen lezen wat er allemaal fout kan gaan met gegevens in operationele databases. Tot op dit moment zijn we elke keer uitgegaan van één operationele database, het CRM-pakket of de ERP-oplossing. De praktijk is dat het helemaal niet blijft bij één systeem. Bijna alle organisaties hebben meer dan één database. Dat levert weer nieuwe uitdagingen op.

Stel: een organisatie heeft een CRM-pakket. Dit is de software waar de verkoopafdeling de hele dag mee werkt. Verkopers gaan langs bij klanten en klanten bellen naar hun contactpersonen. Alle aantekeningen over gemaakte afspraken worden hierin bijgehouden. Maar ook uitgebrachte offertes staan hierin met hun status. Een deel van die offertes zullen orders worden die vervolgens gefactureerd worden.

Naast dit CRM-pakket is er een financieel pakket binnen de organisatie. Dit is het pakket waarmee de financiële administratie en de boekhouding werken. Hier worden herinneringen gemaakt, betalingen verwerkt en kosten geboekt onder de juiste grootboekrekeningen.

Uiteraard zijn beide pakketten voorzien van standaardrapportages om de primaire processen in de gaten te houden. Beide pakketten hebben een omzetrapportage. Bij het CRM-pakket is dit gebaseerd op orders en de status van die orders. Het financiële pakket baseert het rapport op ontvangen betalingen. Vaak zal blijken dat de cijfers op beide rapporten niet overeenkomen. In plaats van over strategie gaat het managementoverleg opeens over wie nu de juiste cijfers heeft en hoe de organisatie er nu eigenlijk voorstaat.

Een deel van dit probleem zal wel eenvoudig op te lossen zijn. Misschien dat het ene rapport factuurdatum als uitgangspunt neemt (btw-afdracht in Nederland is gebaseerd op factuurdatum) terwijl het andere rapport de datum van betalen als uitgangspunt neemt. Aangezien tussen factureren en betaald krijgen een nogal fors verschil kan zitten, zullen beide rapporten andere cijfers tonen.

■ Als twee verschillende systemen dezelfde cijfers zouden moeten geven, zal er in de praktijk vaak een verschil tussen beide systemen te zien zijn.

Ook andere definitie-achtige verschillen kunnen optreden. Wat is nu precies omzet? Is dat inclusief of exclusief btw? Of bijvoorbeeld cost-of-sales. Wat valt daar allemaal wel en niet onder?

In paragraaf 3.1 hebben we gezegd dat een datawarehouse een '*single version of the truth*' moet zijn. In het bovenstaande zijn er twee databases en daarmee ook twee versies van de waarheid. Dat leidt tot verwarring. En dat leidt ertoe dat het managementoverleg gaat over de verkeerde zaken.

Eén database biedt de mogelijk met één definitie te komen van zaken als omzet en cost-of-sales. Alle rapporten die zich baseren op deze ene bron, tonen nu dezelfde informatie. Welke definitie gekozen wordt is eigenlijk niet van belang. Zolang als er consequent mee gewerkt wordt, hebben wij de gewenste eenduidigheid.

■ Eén centraal datawarehouse zorgt voor het gebruik van één centrale definitie van termen zoals omzet en cost-of-sales.

Welke definitie gekozen wordt, is niet van belang voor de Business Intelligence-consultant. De gekozen definitie is een business rule (bedrijfsregel), een keuze van de organisatie. Het is van groot belang om tijdens de informatieanalyse te achterhalen of er meer dan één definitie gebruikt wordt binnen de organisatie. Er moet dan een discussie volgen over de gewenste definitie voor de BI-oplossing. Dat klinkt gemakkelijk, maar kan een erg lastige discussie opleveren met veel politieke en op sentiment gebaseerde argumenten. Voor een Business Intelligence-oplossing is maar één ding echt van belang: eenduidigheid. En dat is wat je met een datawarehouse kunt bieden.

■ Eenduidigheid van gegevens en definities is van cruciaal belang voor Business Intelligence-oplossingen.

3.7 Historische gegevens

Helemaal in het begin van dit hoofdstuk heb ik een lijst van zaken opgenoemd die je kunt tegenkomen als je een rapportage/analyse-omgeving bouwt rechtstreeks op operationele database. Het laatste punt was: Geschiedenis gaat vaak verloren in operationele databases.
Bekijk eens de onderstaande twee tabellen met verkopers en omzet:

VerkoperID	Verkoper	Regio
1	Jan	Noord
2	Piet	Zuid

VerkoperID	KlantID	Datum	Omzet
1	1	10-jun-2013	€ 1500,–
1	2	18-jun-2013	€ 2500,–
2	3	11-jun-2013	€ 1000,–
2	4	20-jun-2013	€ 750,–

Met deze gegevens kun je een analyse doen voor omzet per regio. De query is redelijk eenvoudig:

```
SELECT
   Regio
   , SUM(Omzet) AS Totaal
FROM
   Verkoper
JOIN Order ON Verkoper.VerkoperID = Order.VerkoperID
GROUP BY Regio
```

Als uitkomst krijg je twee records: één met € 4000,– omzet in regio Noord en één met € 1750,– in Zuid. Maar wat nu als Jan in regio Zuid gaat werken? Hoe gaat de database om met deze wijziging?

In veel operationele databases houdt het wijzigen van bestaande records in dat de oude, niet meer relevante gegevens, overschreven worden met de nieuwe actuele gegevens. In dat geval komt de tabel *Verkoper* er als volgt uit te zien:

VerkoperID	Verkoper	Regio
1	Jan	Zuid
2	Piet	Zuid

Als je de omzet per regio query nog een keer uitvoert, blijkt dat er helemaal geen omzet is behaald in de regio Noord. Alle omzet is gescoord in regio Zuid. De omzet van Jan is mee verhuisd naar het zuiden! Dit probleem, dat ontstaat doordat gegevens over-

schreven worden door meer actuele gegevens, is door Ralph Kimball benoemd. Je hebt hier te maken met een Slowly Changing Dimension (SCD). Zie het volgende hoofdstuk voor meer over Kimball en Slowly Changing Dimensions.

Het is duidelijk dat als een operationeel systeem op deze manier wijzigingen doorvoert, de analysemogelijkheden ernstig beperkt worden. Wederom kan een datawarehouse een oplossing bieden. Het volgende hoofdstuk gaat uitgebreid in op mogelijke oplossingen voor dit probleem.

■ Operationele databases houden vaak geen historie bij met betrekking tot wijzigingen. In een datawarehouse kun je dit wel doen. Datawarehouses bieden daardoor uitgebreidere analysemogelijkheden dan operationele databases.

3.8 Conclusie

Een Business Intelligence-oplossing moet mensen van zinvolle en betrouwbare informatie voorzien met als doel betere besluiten te kunnen nemen. Daar waar we die informatie uit operationele systemen halen, zijn er veel factoren die zowel het aspect 'zinvol' als het aspect betrouwbaar negatief beïnvloeden. Het bouwen van een datawarehouse mag nooit een doel op zichzelf worden. Een datawarehouse biedt een (mogelijke) oplossing voor de in dit hoofdstuk beschreven zaken om zo het doel van Business Intelligence te kunnen bereiken.

Het zal na het lezen van dit hoofdstuk duidelijk zijn dat er veel mogelijke problemen kunnen optreden als je uitgebreide rapportages en analyses gaat doen op operationele databases. De gegevens zullen bewerkt moeten worden voordat ze tot zinvolle en betrouwbare informatie leiden.

Een datawarehouse speelt een centrale rol in dit geheel. In de hoofdstukken 4 en 5 kun je uitgebreid lezen over hoe je een datawarehouse kunt opzetten. Na dit hoofdstuk moet duidelijk zijn waarom je dat zou willen.